



Docker Compose

Multi-Container Communication Tutorial

Networking

Services

Volumes

Compose File

What you'll learn:

- ▶ What Docker Compose is and why it matters
- ▶ How to define multi-container apps in a YAML file
- ▶ Container-to-container networking and service discovery
- ▶ Hands-on examples with web server, API, and database

What is Docker Compose?



Definition

Docker Compose is a tool for defining and running multi-container Docker applications.

Using a single YAML file (docker-compose.yml), you declare all services, networks, and volumes your application needs.

Key Facts



One command:

docker compose up starts everything



Declarative:

Describe the desired state in YAML



Isolated:

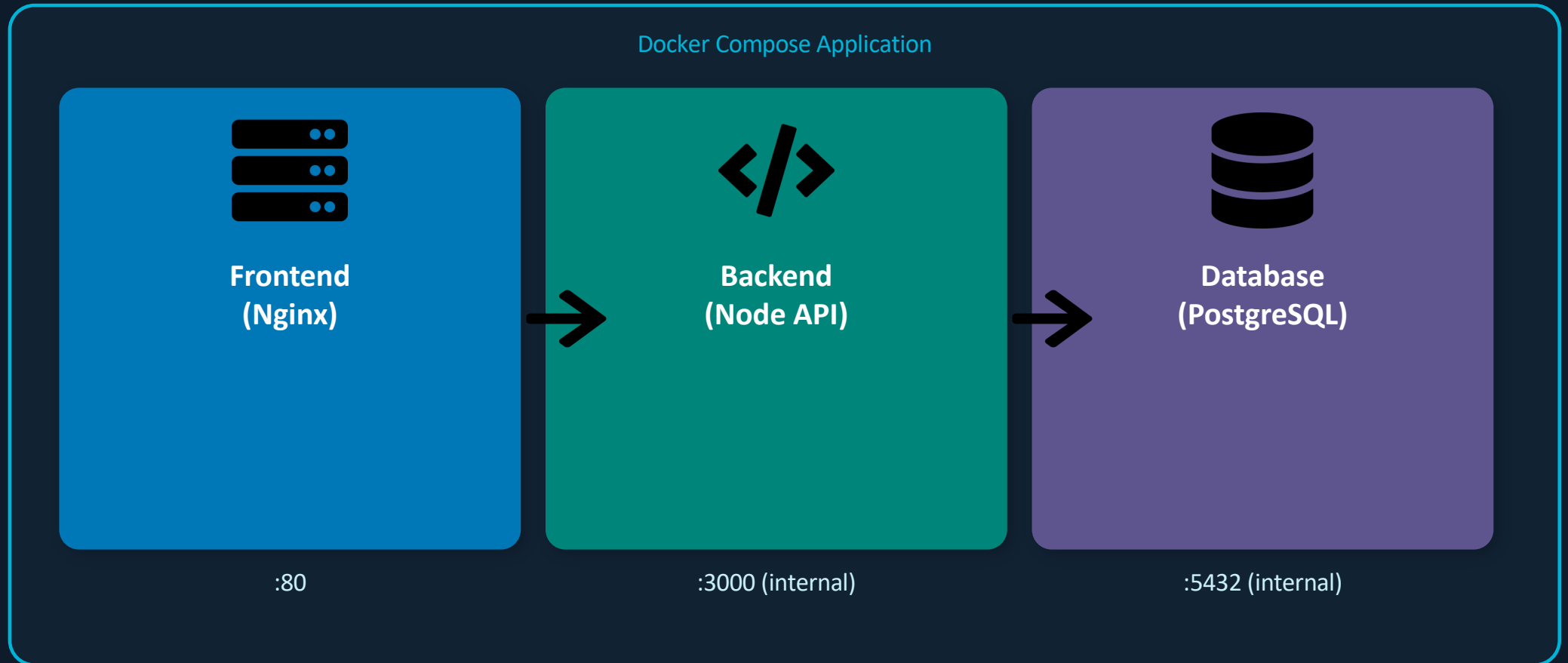
Each service runs in its own container



Networked:

Services auto-discover each other by name

Architecture: Multi-Container Setup



The docker-compose.yml File

```
version: '3.9'

services:
  frontend:
    image: nginx:alpine
    ports:
      - "80:80"
    depends_on: [backend]

  backend:
    build: ./backend
    environment:
      DB_HOST: db

  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
```

services: Each key is a container

image / build: Use a pre-built image or build from Dockerfile

ports: Map host_port:container_port

environment: Inject env vars into the container

depends_on: Control startup order

volumes: Persist data beyond container restarts

Networking & Service Discovery



How Containers Talk to Each Other

Docker Compose automatically creates a shared network for all services. Each service is reachable by its name — no IPs required.

frontend

my_app_default network

db

```
# In your backend app code — connect to db by service name
DB_HOST="db"      # resolves to the db container's IP automatically
FRONTEND_API_URL="http://backend:3000"  # service name as hostname
```

Essential CLI Commands

```
docker compose up -d
```

Start all services in detached (background) mode

```
docker compose ps
```

List running services and their status

```
docker compose build
```

Build or rebuild images from Dockerfiles

```
docker compose restart db
```

Restart a single service

```
docker compose down
```

Stop and remove containers, networks

```
docker compose logs -f
```

Follow real-time logs for all services

```
docker compose exec backend sh
```

Open a shell inside a running container

```
docker compose down -v
```

Also remove named volumes (wipe data)

Volumes & Data Persistence

Named Volumes

Managed by Docker. Data persists after containers stop or are removed.

Best for: databases, uploaded files.

```
volumes:
  pgdata: # declare here

services:
  db:
    volumes:
      - pgdata:/var/lib/postgresql/data
```

Volumes survive docker compose down!

Use docker compose down -v to also delete volumes.

Bind Mounts

Map a host directory directly into a container.

Best for: local development, hot-reload.

```
services:
  backend:
    volumes:
      # host_path:container_path
      - ./backend:/app
      - /app/node_modules
```

Changes to `./backend` reflect instantly inside the container.

Environment Variables & Configuration

Inline in Compose

```
environment:  
  NODE_ENV: production  
  PORT: 3000
```

Simple but exposed in your git repo

.env File

```
# .env (git-ignored)  
POSTGRES_PASSWORD=secret  
APP_PORT=3000
```

Best practice for secrets — add to
.gitignore

env_file directive

```
env_file:  
  - .env  
  - .env.local
```

Load multiple env files into a service

⚠ Never commit secrets (passwords, API keys) to git. Use .env files excluded via .gitignore, or a secrets manager in production.

> Hands-On Lab: Web + API + Database

1 Clone the starter repo

```
git clone https://github.com/example/compose-lab
&& cd compose-lab
```

2 Review the project structure

```
ls -la
# frontend/  backend/  docker-compose.yml
.env.example
```

3 Copy and edit env file

```
cp .env.example .env
# Edit .env: set POSTGRES_PASSWORD=yourpassword
```

4 Build & start all services

```
docker compose up --build -d
```

5 Check everything is running

```
docker compose ps
docker compose logs backend
```

6 Test inter-container communication

```
# Open http://localhost:80
# API: http://localhost:80/api/health
```

Best Practices & Key Takeaways



One service per container

Keep each container focused on a single responsibility.



Use named volumes for data

Never store persistent data inside containers.



Leverage service names

Connect services using their name as the hostname — never hardcode IPs.



Keep secrets in .env

Use environment files; never commit credentials to source control.



Use `depends_on` wisely

Pair with healthchecks for reliable startup ordering.



Use custom networks

Isolate service groups for better security and clarity.